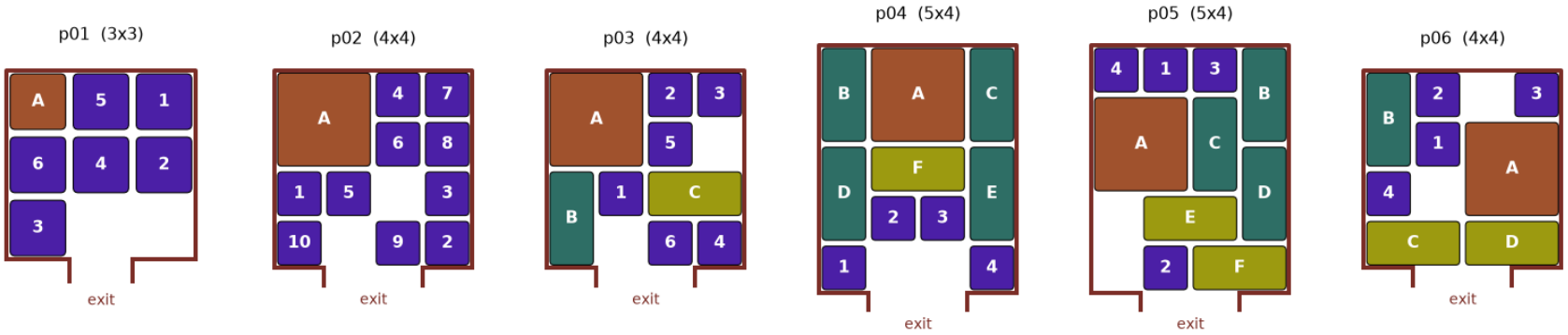
Generalized Klotski

Lab 4: path search

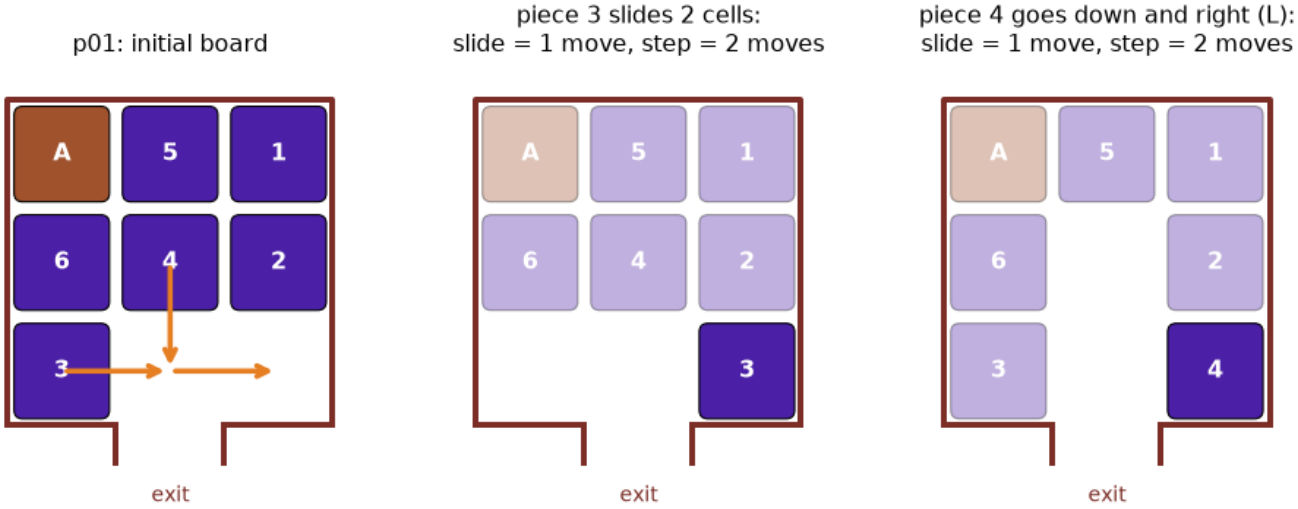
Computational Intelligence 2026-27

The problem



- Six boards with rectangular pieces and two empty cells
- Goal: the target piece A covers the exit cells on the bottom edge
- Wanted: a path, the sequence of moves, as short as possible

What counts as one move

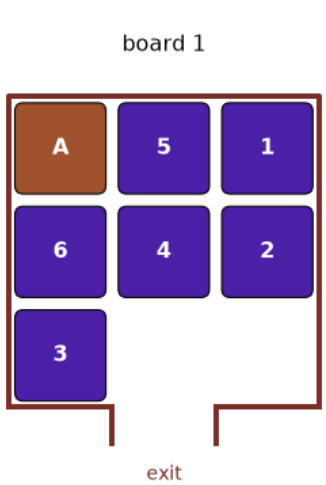


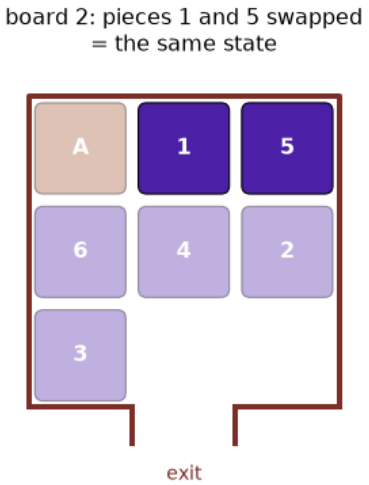
- **Slide:** one piece moves any number of cells, also around a corner. Cost 1
- **Step:** one cell is one move. Implemented for comparison

The state

```
p01, initial state

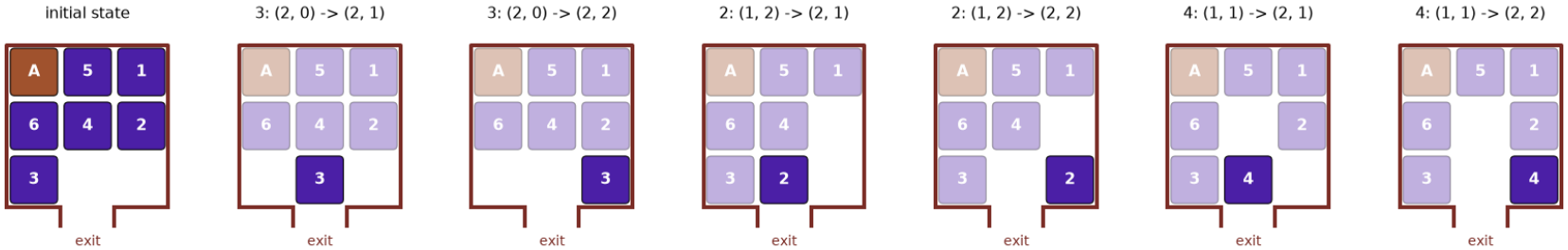
(
 ((0, 0), (0, 0)),   # target
 ((1, 1), (0, 1)),
 ((1, 1), (0, 2)),
 ((1, 1), (1, 0)),
 ((1, 1), (1, 1)),
 ((1, 1), (1, 2)),
 ((1, 1), (2, 0)),
)
```





- Each piece is (kind, top-left cell). The kind is its shape, or a special value for the target
- No names: swapping two equal pieces gives the same state, so it is explored once
- Sorted tuple: one writing per state
- Target_shape is saved in the Problem istance

The moves




```
targets.add((r, c))
todo = [(r, c)]
while len(todo) > 0:
    (pr, pc) = todo.pop()
    for position in one_cell(pr, pc, h, w, free, problem):
        if position not in targets:
            targets.add(position)
            todo.append(position)
targets.discard((r, c))      # staying still is not a move
```

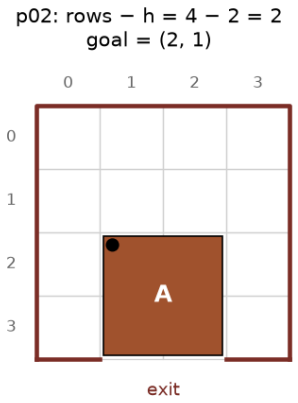
one_cell: where the piece can go with a one-cell shift (inside the board, free cells only)
free_cells: the empty cells + the cells of the piece itself
moves: all the states reachable with one move. For each piece, every position it can reach by repeated one-cell shifts → one new state per position

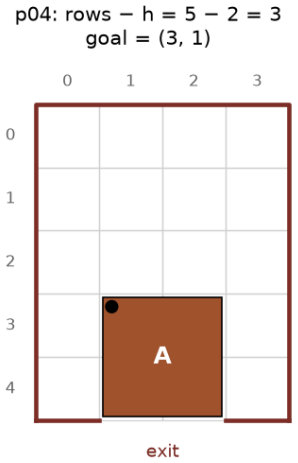
The goal test

```
def target_goal(problem):
    h = problem.target_shape[0]
    return (problem.rows - h, problem.exit_cols[0])

def target_pos(state):
    for (kind, position) in state:
        if kind == TARGET:
            return position

def is_goal(state, problem):
    return target_pos(state) == target_goal(problem)
```





- The target must touch the bottom edge, above the exit columns: its top-left cell must be (rows − h, first exit column)
- A test, not a list of goal states: the other pieces can be anywhere
- The test is done when a state is extracted

Generic search

```
while len(frontier) > 0:
    entry = heapq.heappop(frontier)
    s = entry[2]
    if s in expanded:          # lazy deletion
        continue
    if is_goal(s, problem):    # on extraction
        return build_path(parent, s)
    expanded.add(s)
    for t in moves(s, problem):
        g = cost[s] + 1
        if t not in cost or g < cost[t]:
            cost[t] = g
            parent[t] = s
            tie = tie + 1
            heapq.heappush(frontier,
                (priority(t, g, len(cost), problem),
                     tie, t))
```

- **Frontier:** a heap of discovered states
- **Lazy deletion:** a cheaper path adds a new entry, the old one is skipped when it comes out
- **Goal test:** when a state is extracted, not when it is discovered
- **Path:** parent links, followed back from the goal

Two heuristics

- Both come from relaxing the problem, so they never overestimate
- **h_distance:** ignore the other pieces. With the slide rule, 1 if the target is not on the exit, because one move may be enough
- **h_blockers:** h_distance + 1 for every other piece on the goal cells, because each one must move at least once
- One move changes h_blockers by at most 1, so it is also consistent

B	**A**	**A**	C
B	**A**	**A**	C
D	F	F	E
D	2	3	E
1			4

p04: grey = goal cells

h = 1 + 2 = 3

pieces 2 and 3 are on the goal cells

real cost: 81 moves

The strategies only change the priority

Strategy	Priority	Shortest path guaranteed?
Breadth-first	order of discovery	Yes, every move costs 1
Depth-first	reverse order of discovery	No
Uniform cost	g	Yes
Greedy best-first	h (distance + blockers)	No
A* (distance)	g + h_distance	Yes
A* (distance + blockers)	g + h_blockers	Yes

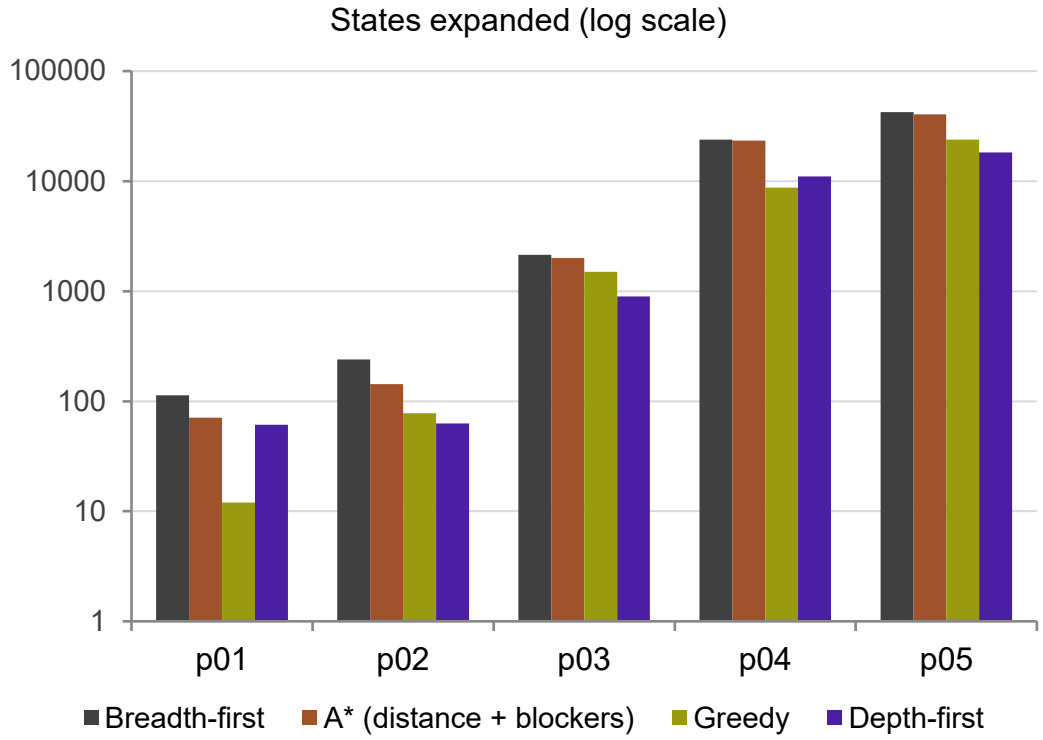
g = cost of the path so far, h = estimate of the moves still needed

Results (slide rule): moves / states expanded

	Shortest	Breadth-first	Uniform cost	A* (distance)	A* (distance + blockers)	Greedy	Depth-first
p01	**6**	6 / 113	6 / 113	6 / 104	6 / 71	6 / 12	58 / 61
p02	**11**	11 / 239	11 / 239	11 / 223	11 / 143	14 / 78	38 / 63
p03	**30**	30 / 2,142	30 / 2,142	30 / 2,115	30 / 2,002	45 / 1,506	626 / 894
p04	**81**	81 / 23,822	81 / 23,822	81 / 23,792	81 / 23,442	94 / 8,753	3632 / 11,070
p05	**138**	138 / 42,473	138 / 42,473	138 / 42,017	138 / 40,387	206 / 23,807	5500 / 18,234
p06	**none**	none / 833	none / 833	none / 833	none / 833	none / 833	none / 833

- Breadth-first, uniform cost and A* always find the shortest solution
- Greedy and depth-first expand fewer states, but find longer solutions

A* saves little over breadth-first



	Saved
p04	1.6%
p05	4.9%

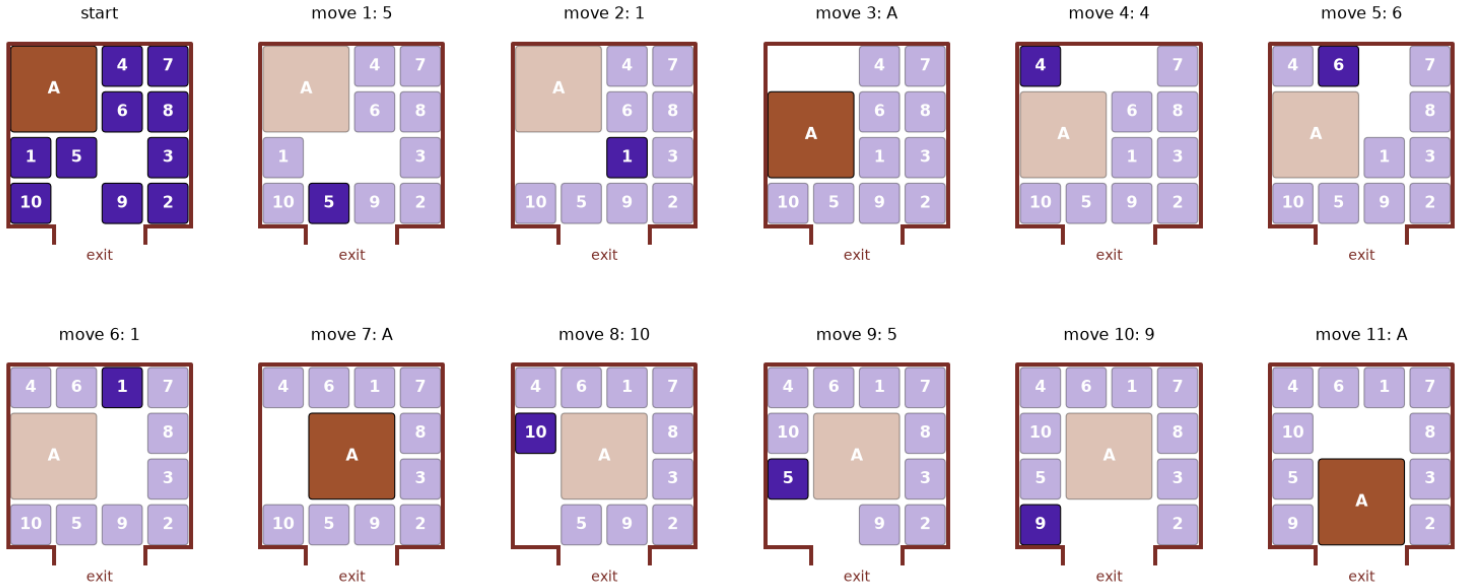
- The heuristics see only the target and the pieces on its goal cells
- Most moves of a solution move the other pieces: h = 3 against 81 on p04
- So g + h is close to g, and A* behaves almost like breadth-first

Both rules

Shortest solution	p01	p02	p03	p04	p05	p06
Slide rule (moves)	6	11	30	81	138	none
Step rule (cells)	8	18	45	116	179	none

- p04 is the classic Klotski: 81 and 116 are its two known values
- p06 has no solution with either rule

A solution: p02 in 11 moves



States have no names: they are given back by following the path from the initial board

Conclusions

- **The state:** no names and one writing per state avoid exploring duplicates
- **Optimal solutions:** 6, 11, 30, 81 and 138 moves, and p06 has none
- **A* is optimal but barely faster:** an admissible heuristic can still be weak
- **Greedy and depth-first:** fewer expansions, longer solutions